

MOG in Clojure
Attack of the clones
(Episode II)

Hello!

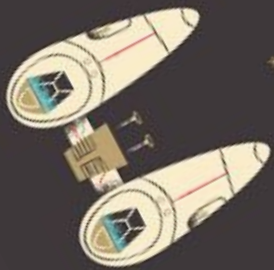
I am **Mey Beisaron**

- Software engineer
- Infra Developer at **FORTER**
- Writing in : Clojure, Nodejs, Groovy, Python
- Public speaker & Mentor
- Sworn Star Wars fan



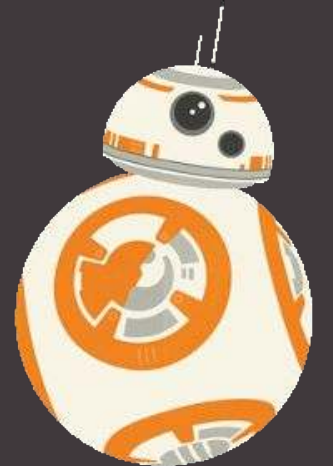


I know why you're
here...





**You want to develop a
MOG!**

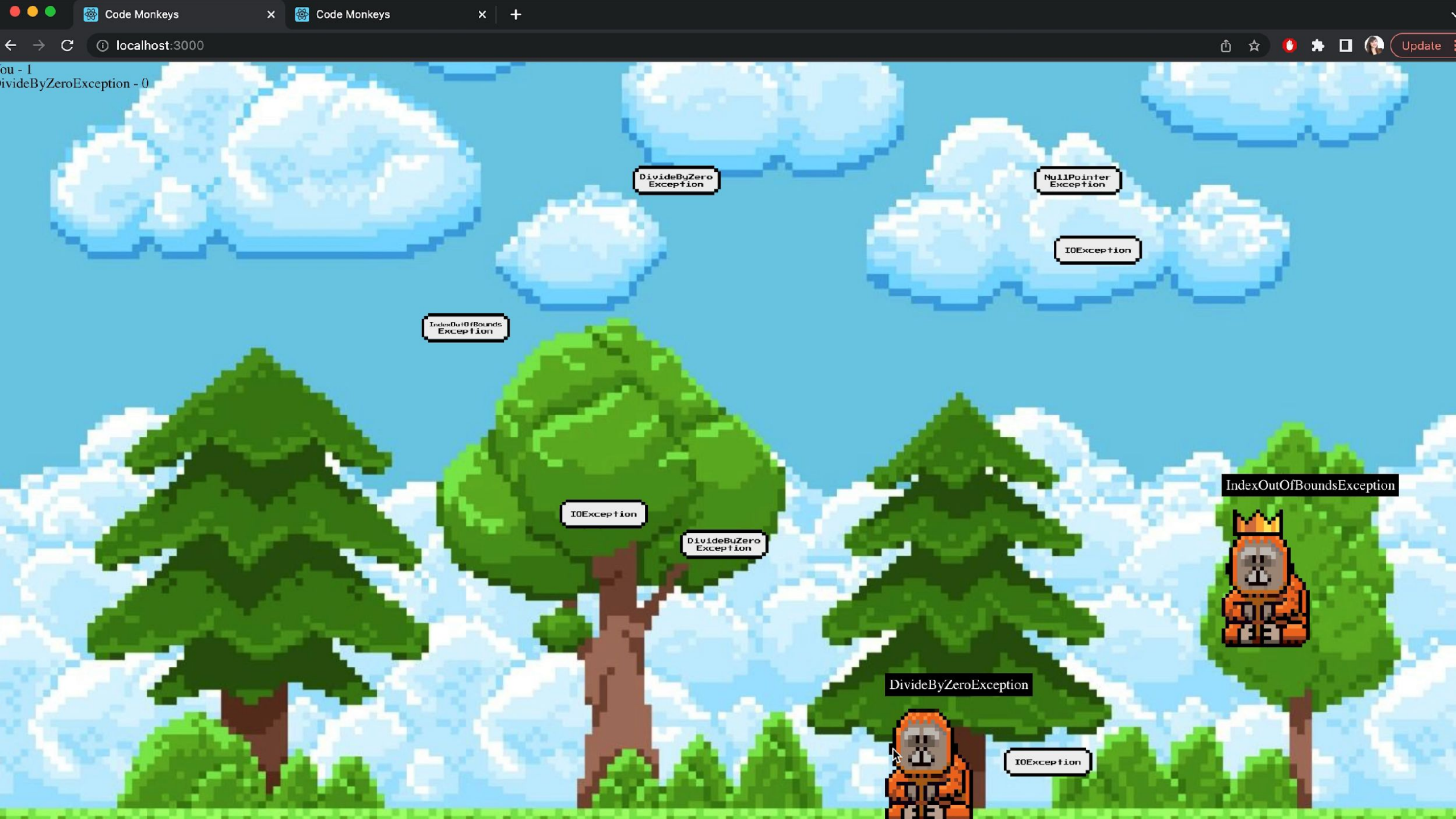




A screenshot from a Star Wars game, likely Star Wars Battlefront II, showing a street battle in a city. In the foreground, a large, dark, mechanical vehicle, possibly a tank or a heavy transport, is partially visible. The street is paved with cobblestones and is littered with debris, including what appears to be a fallen figure. In the background, there are trees with green foliage and buildings with a Mediterranean or Spanish architectural style. A large plume of white smoke or steam rises from the right side of the street, partially obscuring the buildings. On the left, there is a bright orange and yellow explosion or fire. Several vertical banners with blue and red stripes hang from the trees. The overall scene depicts a chaotic battle environment.

it won't be
Star Wars Battlefront II





DivideByZeroException

NullPointerException

IOException

IndexOutOfBoundsException

IOException

DivideByZeroException

IndexOutOfBoundsException

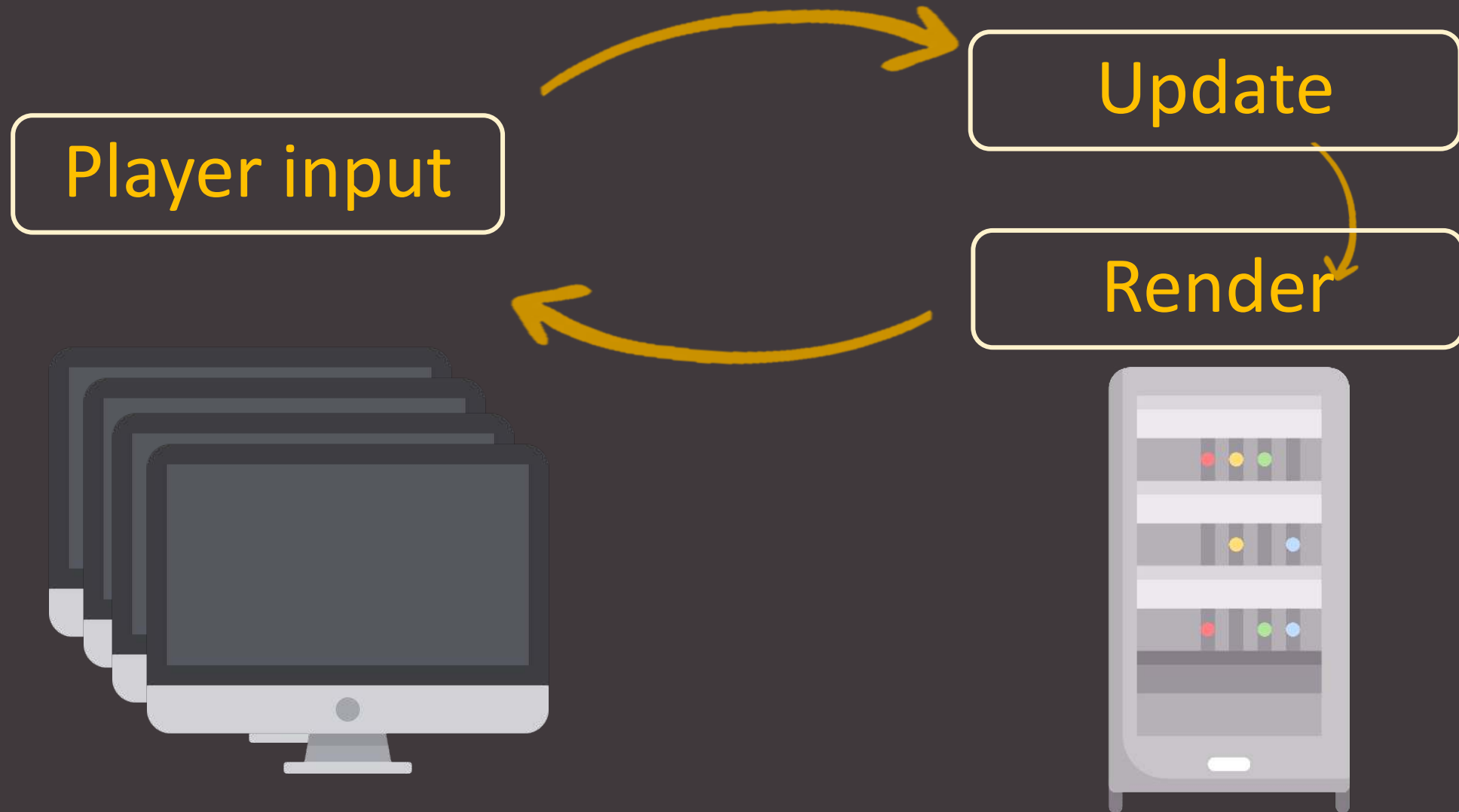
DivideByZeroException

IOException

The Game Loop



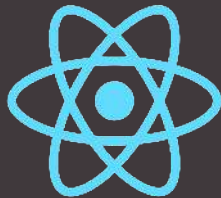
The Game Loop (online)



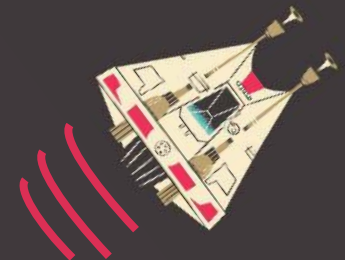
Game Architecture

Websockets over TCP/IP

Websockets API



Websockets API



Server Architecture



Websockets API

Network Communication
Functions

Game State Functions

Game Entities Data Structures



@Ladymey

Server Architecture



Websockets API

Network Communication
Functions

Game State Functions

Game Entities Data Structures

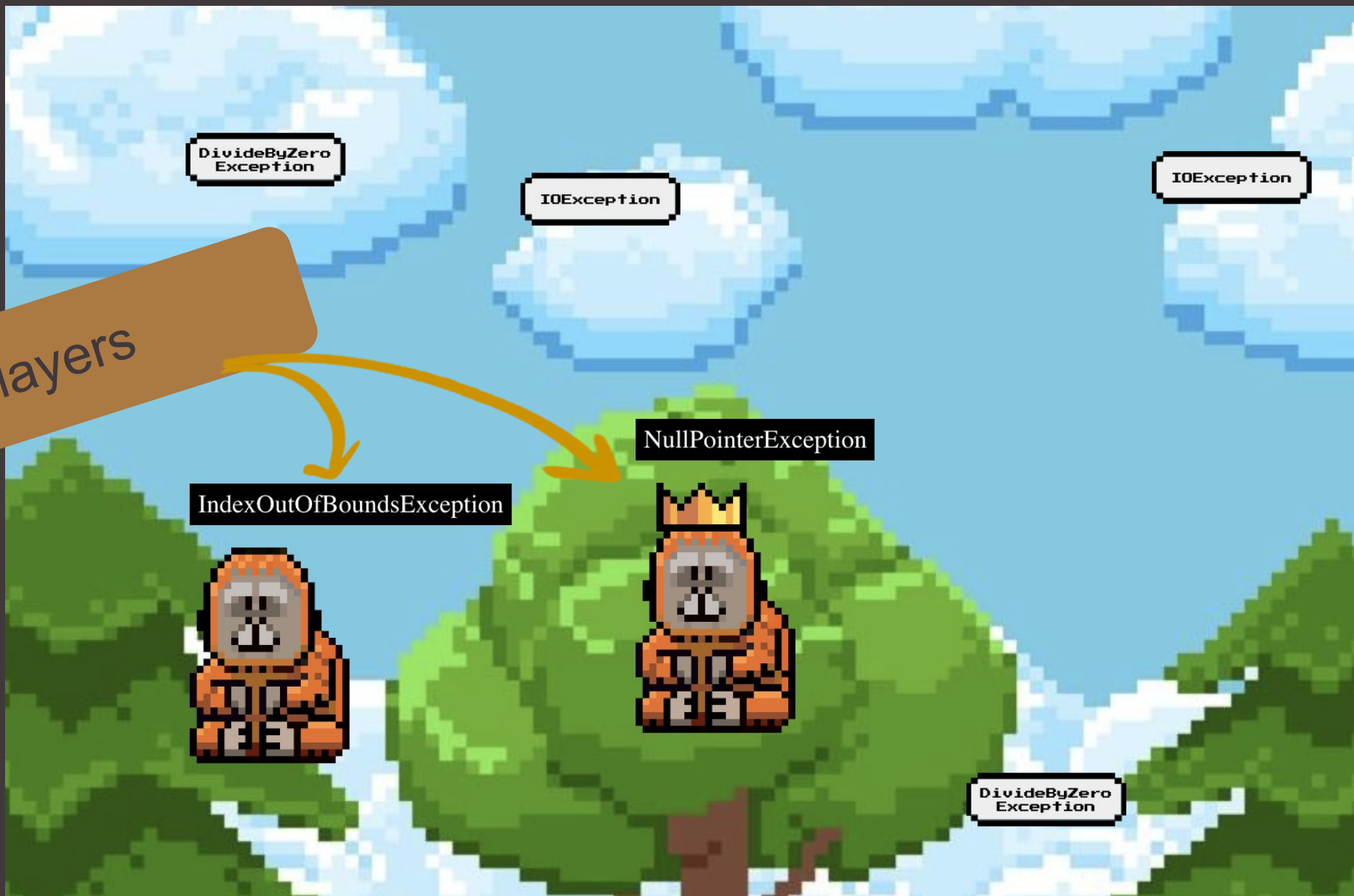


@Ladymey

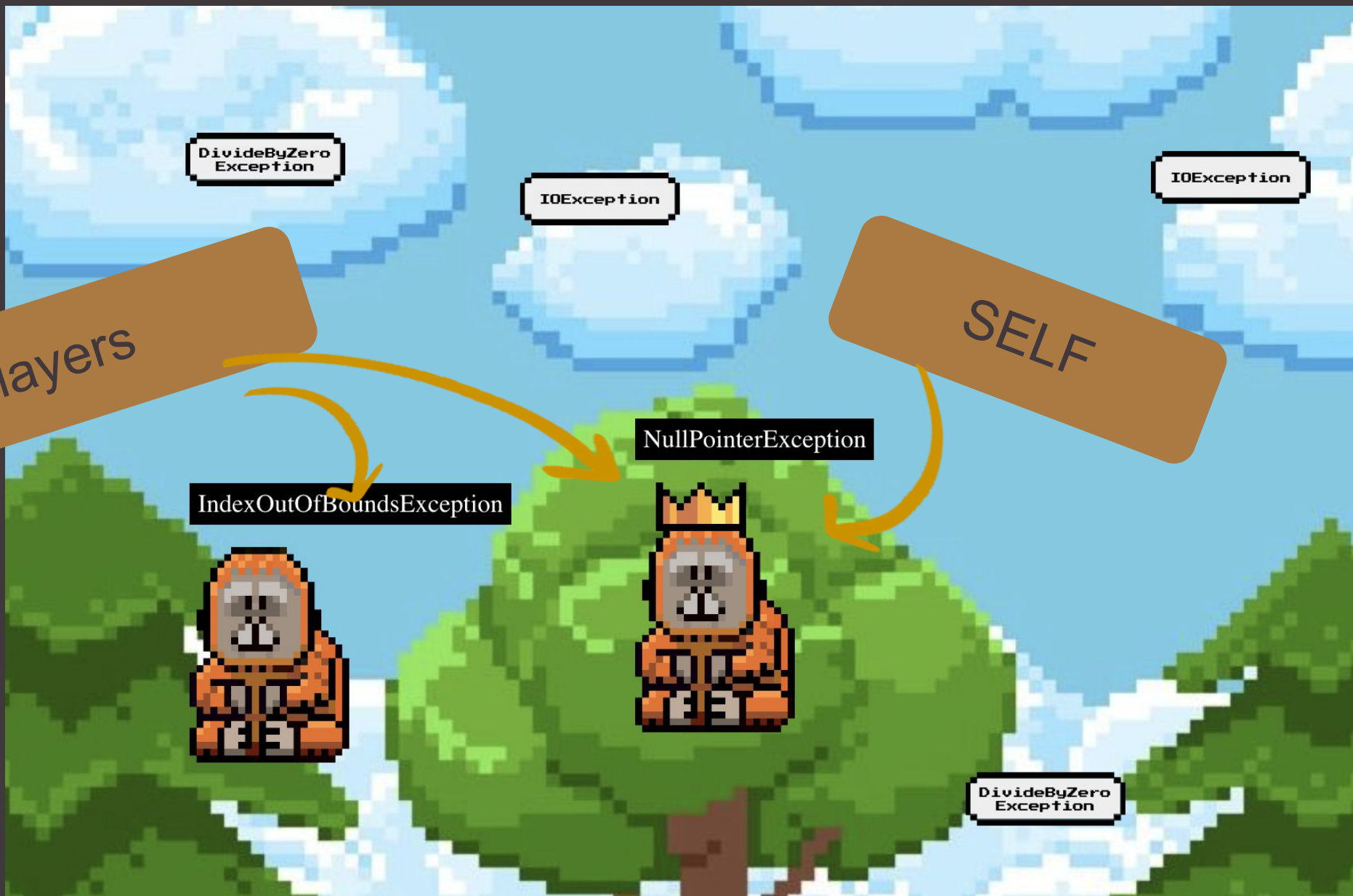
Collectable Items



@Ladymey



@Ladymey



@Ladymey

Game Entities Data Structures

```
(def exceptionTypes ["IOException", "DivideByZeroException",  
                    "NullPointerException", "IndexOutOfBoundsException"])  
(def items (atom {}))  
(def players (atom {}))
```



Game Entities Data Structures

```
(def exceptionTypes ["IOException", "DivideByZeroException",  
                    "NullPointerException", "IndexOutOfBoundsException"])  
(def items (atom {}))  
(def players (atom {}))
```



Game Entities Data Structures

```
(def exceptionTypes ["IOException", "DivideByZeroException",  
                    "NullPointerException", "IndexOutOfBoundsException"])  
(def items (atom {}))  
(def players (atom {}))
```



Game Entities Data Structures

```
(def exceptionTypes ["IOException", "DivideByZeroException",  
                    "NullPointerException", "IndexOutOfBoundsException"])  
(def items (atom {}))  
(def players (atom {}))
```



Game Entities Data Structures

```
(def exceptionTypes ["IOException", "DivideByZeroException",  
                    "NullPointerException", "IndexOutOfBoundsException"])  
(def items (atom {}))  
(def players (atom {}))
```

```
[ :connection-map  
  { :player?      true  
    :id           "356592e8-f30d-4cdc-9751-2988f6236e04"  
    :x            0.65  
    :y            0.5  
    :score        0  
    :show         true  
    :exceptionType "NullPointerException"  
    :collision     false}  
]
```

Game Entities Data Structures

```
[ :connection-map
```

```
  { :player?      true
    :id           "356592e8-f30d-4cdc-9751-2988f6236e04"
    :x            0.65
    :y            0.5
    :score        0
    :show         true
    :exceptionType "NullPointerException"
    :collision     false}
]
```



Game Entities Data Structures

```
[ :connection-map
  { :player?      true
    :id           "356592e8-f30d-4cdc-9751-2988f6236e04"
    :x            0.65
    :y            0.5
    :score        0
    :show         true
    :exceptionType "NullPointerException"
    :collision     false}
]
```



Server Architecture



Websockets API

Network Communication
Functions

Game State Functions

Game Entities Data Structures



@Ladymey

Manage Game State



```
(defn update-game-state [connection player-steps]
  (if (contains? @players connection)
      (move-and-collect connection (:stepX player-steps) (:stepY player-steps))
      (add-new-player
        (get-new-player) connection)))
```



Manage Game State



```
(defn update-game-state [connection player-steps]
  (if (contains? @players connection)
    (move-and-collect connection (:stepX player-steps) (:stepY player-steps))
    (add-new-player
     (get-new-player) connection)))
```



Manage Game State



```
(defn update-game-state [connection player-steps]
  (if (contains? @players connection)
      (move-and-collect connection (:stepX player-steps) (:stepY player-steps))
      (add-new-player
        (get-new-player) connection)))
```



Manage Game State



```
(defn update-game-state [connection player-steps]
  (if (contains? @players connection)
      (move-and-collect connection (:stepX player-steps) (:stepY player-steps))
      (add-new-player
        (get-new-player) connection)))
```



Manage Game State



```
(defn update-game-state [connection player-steps]
  (if (contains? @players connection)
      (move-and-collect connection (:stepX player-steps) (:stepY player-steps))
      (add-new-player
        (get-new-player) connection)))
```



Manage Game State

1. Add new player
2. Player movement
3. Player left the game
4. Handle collectable items



Manage Game State

1. Add new player
2. Player movement
3. Player left the game
4. Handle collectable items

```
(defn get-new-player []  
  {:player?      true  
   :id           (str (uuid/v1))  
   :x            (rand)  
   :y            (rand)  
   :score        0  
   :show         true  
   :exceptionType (rand-nth exceptionTypes)  
   :collision     false})
```



Manage Game State

1. Add new player
2. Player movement
3. Player left the game
4. Handle collectable items



```
(defn add-new-player [player connection]
  (send-msg connection (assoc player :self? true))
  (doseq [existing-player (vals @players)] (send-msg connection existing-player))
  (doseq [existing (vals @items)] (send-msg connection existing))
  (broadcast-msg player)
  (update-player-in-map connection player))
```



Manage Game State



```
(defn add-new-player [player connection]
  (send-msg connection (assoc player :self? true))
  (doseq [existing-player (vals @players)] (send-msg connection existing-player))
  (doseq [existing (vals @items)] (send-msg connection existing))
  (broadcast-msg player)
  (update-player-in-map connection player))
```



Manage Game State

Self -> Client

```
(defn add-new-player [player connection]
  (send-msg connection (assoc player :self? true))
  (doseq [existing-player (vals @players)] (send-msg connection existing-player))
  (doseq [existing (vals @items)] (send-msg connection existing))
  (broadcast-msg player)
  (update-player-in-map connection player))
```



Manage Game State

All existing players -> Client

```
(defn add-new-player [player connection]
  (send-msg connection (assoc player :self? true))
  (doseq [existing-player (vals @players)] (send-msg connection existing-player))
  (doseq [existing (vals @items)] (send-msg connection existing))
  (broadcast-msg player)
  (update-player-in-map connection player))
```



Manage Game State

All existing items -> client

```
(defn add-new-player [player connection]
  (send-msg connection (assoc player :self? true))
  (doseq [existing-player (vals @players)] (send-msg connection existing-player))
  (doseq [existing (vals @items)] (send-msg connection existing))
  (broadcast-msg player)
  (update-player-in-map connection player))
```



Manage Game State

New player-> All clients

```
(defn add-new-player [player connection]
  (send-msg connection (assoc player :self? true))
  (doseq [existing-player (vals @players)] (send-msg connection existing-player))
  (doseq [existing (vals @items)] (send-msg connection existing))
  (broadcast-msg player)
  (update-player-in-map connection player))
```



Manage Game State



```
(defn add-new-player [player connection]
  (send-msg connection (assoc player :self? true))
  (doseq [existing-player (vals @players)] (send-msg connection existing-player))
  (doseq [existing (vals @items)] (send-msg connection existing))
  (broadcast-msg player)
  (update-player-in-map connection player))
```

Manage Game State

1. Add new player
2. Player movement
3. Player left the game
4. Handle collectable items



```
(defn move-and-collect [connection stepX stepY]
  (-> (move-player (@players connection)
                    stepX stepY connection)
      (collect-item connection)
      (broadcast-msg)))
```



Manage Game State



```
(defn move-and-collect [connection stepX stepY]
  (-> (move-player (@players connection) stepX stepY connection)
    (collect-item connection)
    (broadcast-msg)))
```



Manage Game State

```
(defn move-and-collect [connection stepX stepY]  
  (-> (move-player (@players connection) stepX stepY connection)  
        (collect-item connection)  
        (broadcast-msg)))
```



Manage Game State

```
(defn move-and-collect [connection stepX stepY]
  (-> (move-player (@players connection) stepX stepY connection)
      (collect-item connection)
      (broadcast-msg)))
```



Manage Game State



```
(defn move-and-collect [connection stepX stepY]
  (-> (move-player (@players connection) stepX stepY connection)
    (collect-item connection)
    (broadcast-msg)))
```

Manage Game State



```
(defn move-and-collect [connection stepX stepY]
  (-> (move-player (@players connection) stepX stepY connection)
    (collect-item connection)
    (broadcast-msg)))
```

Manage Game State

1. Add new player
2. Player movement
3. Player left the game
4. Handle collectable items

Manage Game State

1. Add new player

2. Player movement

3. Player left the game

4. Handle collectable items

- Delete the player entity
- Update all players with the new game state



```
(defn remove-player [connection]
  (let [player (@players connection)]
    (swap! players dissoc connection)
    (broadcast-msg (assoc player :show false))))
```

Manage Game State

1. Add new player
2. Player movement
3. Player left the game
4. Handle collectable items
 - Generate collectables

Manage Game State

1. Add new player
2. Player movement
3. Player left the game
4. Handle collectable items
 - Generate collectables
 - Detect collisions

Manage Game State

1. Add new player

2. Player movement

3. Player left the game

4. Handle collectable items

- Generate collectables
- Detect collisions
- Handle when collected:
 - Update game state (players map, items map)
 - Update all players with the new game state

Manage Game State

1. Add new player
2. Player movement
3. Player left the game
4. Handle collectable items - Detect collisions



```
(defn collision? [playerX playerY itemX itemY]
  (and (< playerX (+ itemX itemWidth))
        (> (+ playerX playerWidth) itemX)
        (< playerY (+ itemY itemHeight))
        (> (+ playerY playerHeight) itemY)))
```



Server Architecture



Websockets API

Network Communication
Functions

Game State Functions

Game Entities Data Structures



@Ladymey

Network Communication Functions



```
(defn send-msg [connection msg]
  (http-server/send! connection
    (json/generate-string msg {:pretty true})))

(defn broadcast-msg [msg]
  (doseq [connection (keys @players)]
    (send-msg connection msg)))
```



Server Architecture



Websockets API

Network Communication
Functions

Game State Functions

Game Entities Data Structures



@Ladymey

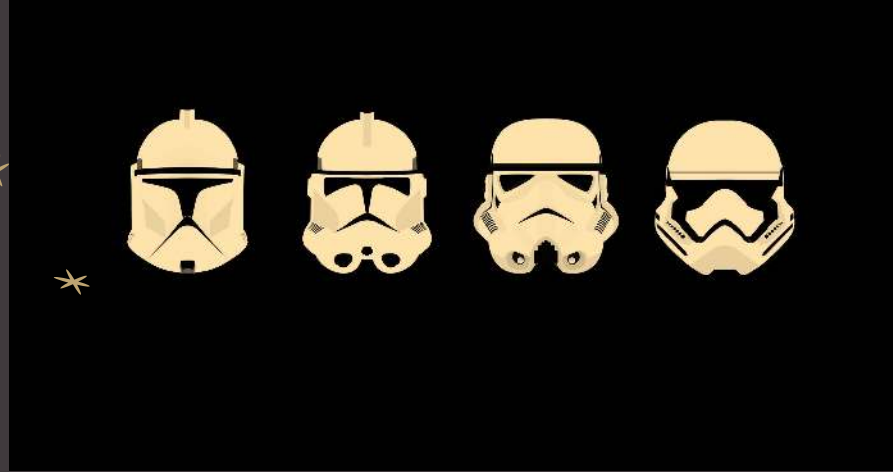
Websockets API



```
(defn ws-handler [request]
  (http-server/with-channel request channel
    (http-server/on-close channel (fn [status]
                                   (println "connection closed:" status)
                                   (remove-player channel)))
    (http-server/on-receive channel (fn [message]
                                      (update-game-state channel message))))))

(def websocket-routes
  (GET "/" [] ws-handler))
```





Summary



@Ladymey

Server Architecture



Websockets API

Network Communication
Functions

Game State Functions

Game Entities Data Structures



@Ladymey

Challenges



@Ladymey



Latency



@Ladymey

Scale







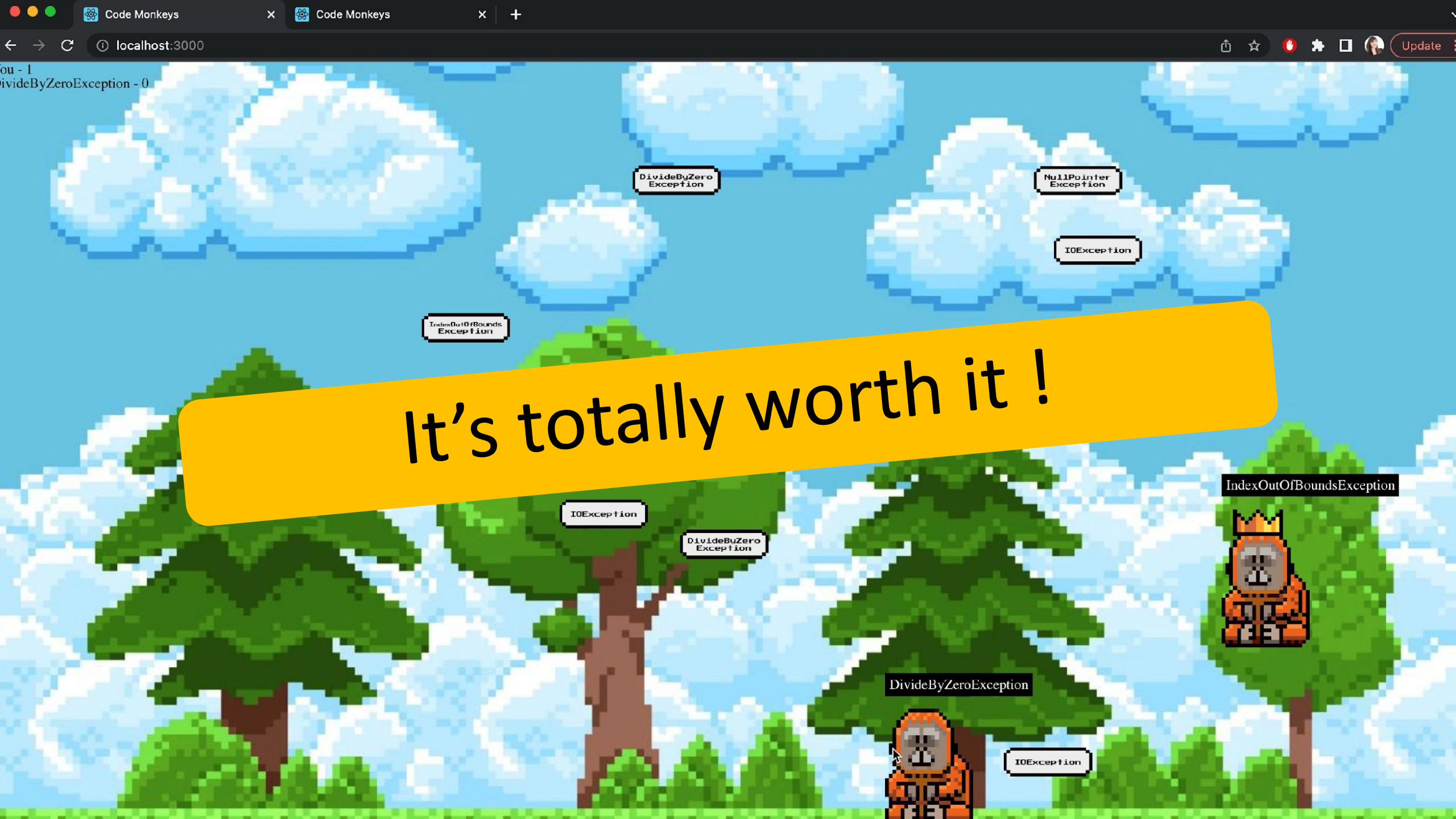
Go develop a MOG!



@Ladymey

A screenshot from a video game, likely Star Wars Battlefront II, showing a street scene. In the foreground, a large, dark, mechanical vehicle, possibly a tank or a heavy transport, is partially visible. The street is paved with cobblestones and has some debris scattered on it. In the background, there are trees with green foliage and several tall, thin poles with flags. The flags are blue and red, and some have gold-colored emblems. There is a lot of white smoke or steam rising from the ground, particularly around the base of the flagpoles and in the distance. On the right side, there is a building with a balcony and some red flowers. The overall atmosphere is one of a battle-torn city.

Although it won't be
Star Wars Battlefront II



It's totally worth it !

DivideByZeroException

NullPointerException

IOException

IndexOutOfBoundsException

IOException

DivideByZeroException

IndexOutOfBoundsException

DivideByZeroException

IOException



Thank you!



@ladymeyy



@ladymey

